Agentic Bug Hunter

Jiwank Shukla (DES DTP DDA AIA)

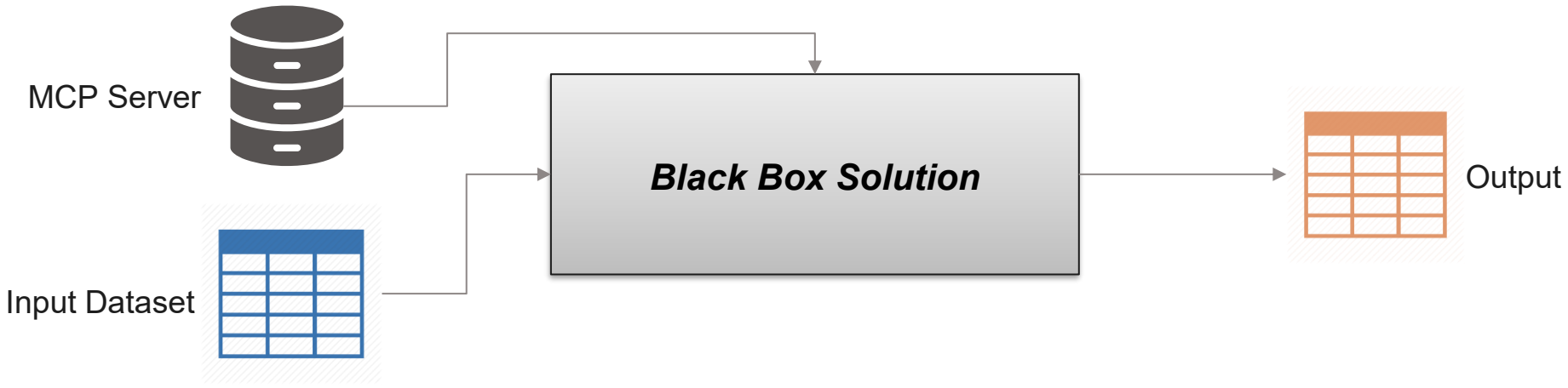
31/01/2026

Problem Statement

Develop an agentic AI system that can seamlessly interact with documentation and script/code to identify errors and provide clear explanation of the bug's nature.

The system must:

1. Pinpoint the exact line containing the bug.
2. Provide a clear explanation of the bug's nature.
3. Use a modular agent design to handle different aspects of bug detection.



Copyright © Infineon Technologies AG 2026. All rights reserved.

Black Box Component

Dataset Table

1. Incorrect Code
2. Correct Code
3. Description

MCP Servers

Data Retrieval Tool

Black Box Solution

Agentic Workflow :

1. Bug detection using retrieved context
2. Bug description using retrieved context
3. Final solution in CSV format

Generated Table

1. Code ID
2. Detected Bug Line
3. Generated Explanation

Copyright © Infineon Technologies AG 2026. All rights reserved.

Objectives and Deliverables

Participants are expected to deliver:

1. **Bug Detection Agent**
   - **Line-Level Precision** : The ability to pinpoint the exact line where the bug first manifests.
   - **Bug Descriptions** : Clear and meaningful explanations of detected bugs, referencing the manual of known bugs.

2. **Modular Agent Collaboration**
   - Collaboration among agents responsible for parsing the code, validating buggy variables, and cross-referencing known bugs using MCP.

Optional Bonus Features:

1. **Interactive Interface**: A GUI or CLI to select C++ files, display detected bugs, and provide summaries.
2. **Code Suggestions**: Automated fixes for simple bugs (e.g., initializing uninitialized variables).

Copyright © Infineon Technologies AG 2026. All rights reserved.

Dataset

Participants will be provided with:

1. A **dataset of C++ code snippets** containing known and unknown faults.
2. **Small context** about the code and it's use.
3. An **Explanation** of bug in the code.
4. A **manual or lookup table** of common bugs, their characteristics, and descriptions. [on MCP Server]

ID	Code	Correct Code	Context	Explanation
0	…	…	RDI method naming	RDI_begin() changed to RDI_END()
1	…	…	RDI burst usage	Changes 'v' value from 3.0 v to -2.0v

Additional Support:
Small information regarding code

Copyright © Infineon Technologies AG 2026. All rights reserved.

Solution Submission Guidelines

- For final submission, participants needs to provide a zipped folder ***("TeamName_Submission.zip" e.g Team1_submission.zip)*** containing following items-

  1. Output file (**output.csv**)
  2. Solution Scripts (**In a Folder name "code"**)
  3. Python libraries used (**requirement.txt**)
  4. Model file (if any)

– **Output file format** The output will be in **CSV format**, consisting of following columns:

ID *(Code ID)* : *Unique ID of the code snippet.*
Bug Line *(Detected Bug Line)* : *Exact line number containing the bug.*
Explanation *(Generated Explanation)* : *Generated explanation of the bug's nature.*

ID	Bug Line	Explanation
0	5	RDI_begin() changed to RDI_END()
1	3	Changes 'v' value from 3.0 v to -2.0v

Copyright © Infineon Technologies AG 2026. All rights reserved.

Evaluation Criteria

- Hybrid Evaluation Concept
  - **Accuracy of Bug Detection (40%)**
  - **Bug Explanations / Reference to mention of bug in documentation (30%)**
  - **Agents or MCP Utilization (30%)**

Copyright © Infineon Technologies AG 2026. All rights reserved.

infineon